

RAPPORT DE PROJET — MAI 2026

Archi-Agent VR

Conception architecturale assistée par IA en réalité virtuelle

Aitelhaj Ghizlane
Farmati Aya
Tahir Aboubakr
Elmerahy khadija
Salma Bouziane
Lyazidi Younes
Abarkha Fatima Ezzahra

Mai 2026

Résumé

Résumé du système

Archi-Agent VR est un assistant d'architecture et de design d'intérieur en réalité virtuelle, piloté entièrement à la voix en français.

L'utilisateur enfle un casque Meta Quest, parle à un avatar IA qui se tient face à lui, et regarde sa maison se construire en temps réel. Le système combine :

- Un client [Unity 2022.3 LTS](#) ;
- Un backend [FastAPI](#) orchestrant un agent LLM ([Llama 3.3 70B](#) sur Groq, avec repli sur [Gemini 2.5 Flash-Lite](#)) ;
- Une persistance [MongoDB](#) ;
- Un pipeline vocal STT-LLM-TTS entièrement bâti sur des paliers gratuits — [Deepgram Nova-3](#), [Groq](#) et [Edge-TTS](#).

Sept capacités majeures sont opérationnelles : contrôle vocal à faible latence (≈ 300 ms), avatar humanoïde animé sur cinq états, mains VR riggées, téléportation vocale, placement intelligent parmi 50 prefabs, tableau de bord analytique en WorldSpace, et génération guidée de maisons. L'architecture privilégie la résilience par cascades à trois niveaux et la simplicité de déploiement (un seul conteneur Docker, une seule scène Unity).

Table des matières

Résumé	1
1 Introduction	3
1.1 Contexte de la conception architecturale en VR	3
1.2 Problématique	3
1.3 Objectifs du système	3
1.4 Périmètre du rapport	4
2 La scène Unity et ses composants	5
2.1 Vue d'ensemble de la scène MaMaison	5

2.2	Éléments fixes : terrain, éclairage, environnement	5
2.3	Le rig joueur	5
2.4	L'avatar IA et son canvas HUD	6
2.5	Le GameManager — services persistants	6
2.6	Catalogue des prefabs	7
2.7	Matériaux et textures	8
3	Architecture du système	9
3.1	Vue d'ensemble	9
3.2	Pipeline vocal en cascade	9
3.2.1	Cascade STT	10
3.2.2	Cascade LLM	10
3.3	Backend FastAPI	10
3.4	Détection de la position du joueur	11
3.4.1	Côté Unity	11
3.4.2	Côté backend — pattern contextvars	11
3.5	Les huit outils de l'agent LLM	12
3.6	RAG local BM25	12
3.7	Persistance MongoDB	13
3.8	Protocole Unity Backend	13
4	Vitrine fonctionnelle	15
4.1	Contrôle vocal en français	15
4.2	Avatar IA animé	15
4.2.1	Machine d'états d'animation	15
4.3	Mains animées dual-mode VR / PC	16
4.4	Téléportation vocale	16
4.5	Placement intelligent d'objets	17
4.6	Tableau de bord analytique	17
4.7	Génération de maison guidée	17
5	Stack technique	19
6	Conclusion	21
6.1	Synthèse des accomplissements	21
6.2	Limites actuelles	21
6.3	Perspectives	21
A	Référence des endpoints REST	22
B	Variables d'environnement	23
C	Structure du dépôt	24

1. Introduction

1.1 Contexte de la conception architecturale en VR

La conception architecturale assistée par ordinateur a longtemps été le monopole de logiciels professionnels — AutoCAD, Revit, SketchUp, ArchiCAD — qui imposent une courbe d'apprentissage de plusieurs mois. La démocratisation des casques grand public (Meta Quest 2 et 3, dès 300 €) couplée à la maturité des grands modèles de langage ouvre une voie alternative : remplacer l'apprentissage d'une interface complexe par une **conversation en langage naturel, immersive et instantanée**.

Ce projet explore concrètement ce qu'il est possible de faire — aujourd'hui, sur du matériel grand public, avec des APIs gratuites — lorsqu'on confie le pilotage d'une scène 3D Unity à un agent LLM côté serveur, relié à un client VR par un protocole de polling robuste sur Wi-Fi.

1.2 Problématique

Trois questions structurent ce travail :

1. Peut-on piloter **intégralement** une scène VR à la voix, sans manettes pour la navigation ni interfaces 2D, en réservant l'usage des mains à la seule manipulation du mobilier ?
2. Un LLM raisonnablement petit (70B, accessible gratuitement sur Groq) suffit-il à orchestrer une scène complexe — placement de prefabs, téléportation, interrogation de l'état du monde, consultation d'une base de normes, gestion d'un historique d'annulation ?
3. Les paliers gratuits (Groq, Deepgram, Gemini, Edge-TTS) sont-ils suffisamment fiables pour une expérience VR immersive, où une coupure de plus de 3 secondes ruine l'immersion ?

La réponse opérationnelle apportée par Archi-Agent VR est **positive** aux trois questions, sous réserve de mécanismes de résilience explicites — cascades à trois niveaux, détection d'effet de bord avant repli, polling tolérant aux coupures Wi-Fi, `contextvars` pour l'injection de position.

1.3 Objectifs du système

— **Langage naturel français** — STT et TTS de qualité, aller-retour voix → action → voix sous 5 secondes.

- **Reproductibilité gratuite** — aucune dépendance payante ; un étudiant peut cloner le dépôt et démarrer sans budget.
- **Retour utilisateur visible** — chaque action produit un effet dans le casque sous 1 seconde.
- **Résilience face aux quotas** — le système continue de fonctionner lors d'un échec ou d'un dépassement de quota.
- **Persistance des sessions** — l'utilisateur retrouve son monde à l'identique au redémarrage.

1.4 Périmètre du rapport

Ce rapport se concentre sur deux axes : l'architecture technique (chapitres 2 et 3) et la vitrine fonctionnelle (chapitre 4). En sont exclus : la chronologie de développement et le guide d'installation, tous deux disponibles dans le README du dépôt GitHub.

2. La scène Unity et ses composants

Avant d'aborder l'architecture logicielle, ce chapitre décrit l'environnement Unity dans lequel l'expérience prend place : la scène MaMaison, ses GameObjects fixes, le rig joueur, l'avatar, les services persistants du GameManager, le catalogue de prefabs et les matériaux.

2.1 Vue d'ensemble de la scène MaMaison

La scène principale (`Assets/Scenes/MaMaison.unity`) est **volontairement minimaliste** à l'ouverture, pour laisser l'IA construire le monde au fil de la conversation. Au démarrage : aucune maison, aucun meuble, aucune clôture. Tout ce qui apparaît ensuite est instancié dynamiquement à l'exécution par `PrefabPlacer` et `LayoutReceiver` à partir des décisions du LLM.

Parti pris de la scène-page-blanche

L'utilisateur démarre dans un environnement neutre où aucun biais visuel ne contraint son imagination. La persistance MongoDB se charge de restaurer la scène précédente au redémarrage.

[Scène MaMaison à l'ouverture — terrain vide, avatar visible au centre, mains du joueur en premier plan]

2.2 Éléments fixes : terrain, éclairage, environnement

Quatre éléments composent la scène vide :

- **Terrain plat** — Composant Terrain Unity, texturé herbe/terre, 500×500 m. Sol pour le raycast de placement.
- **Lumière directionnelle** — Simule le soleil (−50° X, 30° Y, intensité 1,0). Projette des ombres dynamiques.
- **Skybox** — Ciel bleu six faces, définit la couleur ambiante.
- **Volume Post-Processing** — URP. Bloom + ambient occlusion légère en mode PC ; désactivé sur Quest pour maintenir 72–90 Hz.

2.3 Le rig joueur

Le GameObject `Player` regroupe tout le dispositif d'entrée, fonctionnant de manière transparente en mode VR ou PC :

- **XR Origin** — Point d’ancrage VR, XR Interaction Toolkit 3.5, compatible Meta Quest 2 et 3.
- **Main Camera** — FOV 60°. Sa position et son orientation sont lues à chaque push-to-talk (`player_x`, `player_z`, `angle`).
- **LeftHand / RightHand** — Chacun porte un **HandPresence** (dual-mode), un **SphereCollider** trigger, un **XRDirectInteractor** et le prefab Animated Hands.
- **FirstPersonController** — WASD en mode PC, auto-désactivé si un casque XR est détecté.

2.4 L’avatar IA et son canvas HUD

L’avatar IA est placé trois mètres devant le joueur sur un GameObject **Avatar**. Sa hiérarchie comprend un **SkinnedMeshRenderer** (mesh Mixamo Ch41, Layer 8), un **Animator** (`AvatarController.controller`) et un script **AvatarStateController** (cinq états).

Une caméra secondaire **AvatarCamera** ne rend que le Layer 8 et écrit son rendu dans une **RenderTexture** 512×512 px. Le canvas **AvatarHUD** (WorldSpace, suit la caméra principale) affiche cette **RenderTexture** via un **RawImage**. Ce procédé contourne l’impossibilité d’utiliser le mode **ScreenSpaceOverlay** en VR stéréoscopique.

2.5 Le GameManager — services persistants

Un GameObject vide **GameManager** héberge tous les services actifs pendant toute la session. Centralisation : même URL de base API, aucun service perdu en cas de destruction d’un sous-objet.

Service	Rôle
VoiceRecorder	Capture push-to-talk, dialogue HTTP avec <code>/api/chat/audio</code> , lecture du MP3 TTS, émission des événements de pipeline.
PrefabPlacer	Polling toutes les 1,5 s de <code>/api/placements/pending</code> . Raycast sol, instanciation, ajout d’interaction VR, ACK serveur.
TeleportReceiver	Polling toutes les 2 s de <code>/api/teleport/pending</code> . Fondu noir, déplacement du Player, fondu retour, ACK.
UICommandPoller	Polling toutes les 2 s de <code>/api/ui/pending</code> . Émet <code>OnOpenDashboard</code> ou <code>OnCloseDashboard</code> .
DashboardController	Active/désactive le canvas dashboard en WorldSpace, fetch les stats depuis <code>/api/analytics/dashboard</code> .
LayoutReceiver	Réception des descriptions de pièces complètes (murs, sols). Instancie les segments de murs.

(suite)

AnalyticsTracker	Journal local d'événements (push-to-talk, latence, erreurs). Envoi périodique au backend.
-------------------------	---

Table 2.1. Services persistants du GameManager

2.6 Catalogue des prefabs

Le système met à disposition de l'IA un catalogue d'une **cinquantaine de prefabs** répartis en sept catégories, stockés dans **Assets/Resources/**. Le mapping est défini dans `spawnable_objects.json`, lu au démarrage et exposé au LLM dans son prompt système.

Catégorie	Variantes	Description
Maisons modulaires	8 modèles	ModularHousePack1. De 30 m ² (cabanon) à 120 m ² (villa). Styles : chalet, plain-pied, étage+balcon, maison en L.
Chaises	6 entrées	Variantes 1–5 + générique. Salle à manger, bois ou métal, hauteur d'assise ≈45 cm.
Tables	6 entrées	Variantes 1–5 + générique. Basses (45 cm), à manger (75 cm), bureau. XRGrabInteractable.
Cuisine	7 appareils	Évier, four, frigo, plan de travail, plaque, micro-ondes, hotte. Chaque appareil = prefab séparé.
Voies & extérieur	2 entrées	Segments de route (5 m, asphalte) et sentiers (graviers).
Clôtures	2 entrées	Sections bois (3 m, 1,2 m de hauteur) et grilles métal (2 m). Placées côte à côte par l'agent.
Luminaires	2 entrées	Lampadaires (2,5 m) et spots. Point Light Unity intégrée ; max 4 simultanées sur Quest.

Table 2.2. Catalogue des prefabs accessibles au LLM

[Vue d'ensemble du catalogue — exemples de maisons, chaises, tables, lampadaires côte à côte]

2.7 Matériaux et textures

Tous les prefabs ont vu leurs matériaux convertis du shader *Autodesk Interactive* au shader **URP/Lit** standard (via *Edit* → *Render Pipeline* → *URP* → *Upgrade Project Materials*).

Les textures de l'avatar Mixamo Ch41 proviennent du FBX d'origine : `Ch41_body.png` (2048×2048 px) et `Ch41_FacialExpressions.png` (1024×1024 px), avec le mode sRGB activé sur les textures diffuses.

Les mains animées utilisent un matériau bleu translucide (URP/Lit, alpha 0,7), évitant que les mains masquent complètement les objets approchés tout en restant clairement visibles.

3. Architecture du système

3.1 Vue d'ensemble

Archi-Agent VR repose sur une architecture **client-serveur strictement séparée**. Le client Unity ne contient aucun modèle d'IA et délègue toutes les décisions à un backend Python.

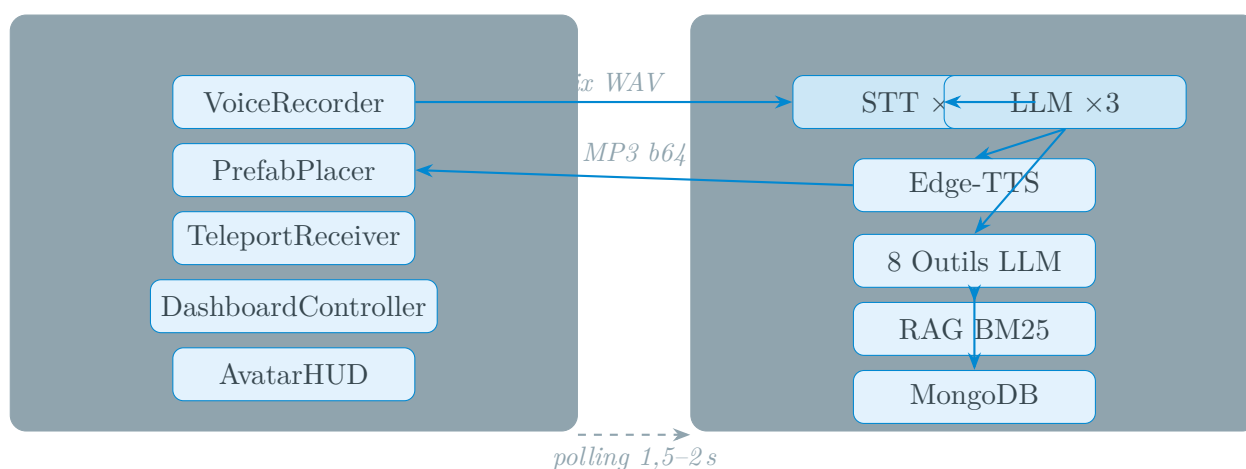


Figure 3.1. Architecture globale du système

Cette séparation offre trois bénéfices : client léger (pas de PyTorch sur Quest), mise à jour du modèle LLM transparente côté Unity, et persistance centralisée (plusieurs casques peuvent partager la même maison).

3.2 Pipeline vocal en cascade

Chaque API critique est servie par une **cascade à trois niveaux**. Si le niveau 1 dépasse son délai ou retourne une erreur (HTTP 429, 5xx), le niveau 2 prend le relais ; si lui aussi échoue, le niveau 3 est sollicité. Chaque niveau utilise une clé ou un fournisseur distinct pour qu'un quota épuisé ne bloque jamais l'ensemble du pipeline.

3.2.1 Cascade STT

Niveau	Service	Modèle	Latence p50	Timeout
1	Deepgram	nova-3 (fr)	≈300 ms	8 s
2	Groq Whisper	whisper-large-v3 (fr)	1–5 s	10 s
3	Gemini	gemini-2.5-flash-lite	2–4 s	défaut

Table 3.1. Cascade de reconnaissance vocale (STT)

Deepgram Nova-3 a remplacé Nova-2 en mai 2026. Nova-2 retournait des chaînes vides sur les phrases courtes (<2 s), forçant systématiquement la cascade à descendre au niveau 2 et dégradant la latence perçue de 300 ms à plusieurs secondes.

3.2.2 Cascade LLM

Niveau	Service	Modèle	Fenêtre TPM	Timeout
1	Groq	llama-3.3-70b-versatile	12 000	15 s
2	Groq	llama-3.1-8b-instant	30 000	15 s
3	Gemini	gemini-2.5-flash-lite	free tier	15 s

Table 3.2. Cascade LLM avec détection d’effet de bord avant repli

Le pipeline LLM intègre une protection critique : la **détection d’effet de bord avant repli**. Si le niveau 1 a déjà appelé l’outil `placer_objet` (chaise insérée dans MongoDB) mais a dépassé son délai pendant la rédaction de sa réponse texte, tomber naïvement sur le niveau 2 placerait une seconde chaise. La fonction `_run_agent_with_fallback` compte les effets de bord avant et après chaque appel ; si le compteur a augmenté, elle court-circuite avec une réponse synthétique.

Pour la synthèse vocale, un seul niveau : **Edge-TTS** (interface Microsoft Azure, gratuite, sans clé). Voix `fr-FR-DeniseNeural`, latence 500–800 ms pour une phrase de dix mots. L’audio est retourné en MP3 base64, décodé côté Unity et lu via un `AudioSource`.

3.3 Backend FastAPI

Le backend est un service Uvicorn unique exposant l’application FastAPI définie dans `api.py`. Quatre familles d’endpoints :

— `/api/chat/audio` — Endpoint principal. Reçoit un WAV multipart, exécute la

cascade STT → LLM → TTS, retourne JSON (transcription + texte + MP3 base64).

- `/api/placements|teleport|ui/pending` — Trois endpoints de polling retournant les commandes non encore livrées à Unity.
- `/api/*/confirm/{id}` — ACK : marque le document MongoDB comme `delivered` après instantiation Unity.
- `/api/analytics/dashboard`, `/api/status` — Statistiques de session et liveness check.

Trois middlewares conditionnent les requêtes : CORS, GZip (le MP3 base64 pèse 30–80 ko — gain significatif sur Wi-Fi Quest), et un récupérateur défensif qui détecte les appels d’outils écrits en clair par le LLM 8b et les exécute côté serveur.

3.4 Détection de la position du joueur

L’agent doit savoir où se trouve le joueur pour interpréter « mets une chaise devant moi ». Cette information est transmise à chaque requête vocale.

3.4.1 Côté Unity

À chaque envoi audio, `VoiceRecorder` ajoute quatre paramètres à l’URL :

```
1 ?house_id=maison_001
2 &player_x=12.45      # Camera.main.transform.position.x
3 &player_z=-3.10      # Camera.main.transform.position.z
4 &player_angle=145.7  # Camera.main.transform.eulerAngles.y
5 &session_id=<uuid>
```

Listing 3.1. Paramètres de position envoyés par Unity

3.4.2 Côté backend — pattern contextvars

Le module `api.py` extrait ces paramètres via une variable `contextvars` (globale par requête, thread-safe) et injecte dans le prompt LLM :

```
1 [POSITION: x=12.5, z=-3.1, angle=146 deg]
2 [BUILD_AHEAD: x=13.7, z=-1.0]
```

Listing 3.2. Bloc de position injecté dans le prompt LLM

`BUILD_AHEAD` est calculé par `spatial.calculate_build_position` :

```
1 # Unity: yaw=0 -> forward = +Z, yaw=90 -> forward = +X
2 # (NON +X comme dans la convention mathématique standard)
3 build_x = player_x + distance * math.sin(math.radians(angle))
4 build_z = player_z + distance * math.cos(math.radians(angle))
5 # ~~~ sin/cos sont INVERSES
```

Listing 3.3. Convention angulaire Unity (sin/cos inversés)

Attention

Cette inversion a coûté une journée entière de débogage. Elle est commentée explicitement dans le code source et couverte par un test unitaire pour éviter toute régression.

3.5 Les huit outils de l'agent LLM

L'agent (bibliothèque `agno`) dispose de huit outils déclarés dans `tools.py`. À chaque tour, le LLM peut en appeler 0, 1 ou plusieurs, dans la limite de 3 appels par requête (`tool_call_limit`) pour éviter les boucles infinies.

Outil	Signature	Effet de bord
<code>placer_objet</code>	<code>(object_name : str)</code>	Insère dans <code>placements</code> . PrefabPlacer Unity instancie.
<code>teleporter_joueur</code>	<code>(destination : str)</code>	Insère dans <code>teleports</code> . TeleportReceiver applique.
<code>obtenir_etat_scene</code>	<code>() / (filter : str)</code>	Lit <code>placements + rooms</code> . Retourne liste structurée.
<code>annuler_derniere_act</code>	<code>()</code>	Dépile l'historique, marque le placement comme supprimé.
<code>obtenir_contexte_analytics</code>	<code>()</code>	Lit les stats de session.
<code>interroger_normes</code>	<code>(question : str)</code>	BM25 sur <code>knowledge_base/*.md</code> (DTU, dimensions).
<code>dashboard</code>	<code>('open' 'close')</code>	Pousse une commande UI dans <code>ui_commands</code> .
<code>position_joueur</code>	<code>()</code>	Retourne coordonnées et angle depuis le <code>contextvar</code> .

Table 3.3. Les huit outils accessibles au LLM

3.6 RAG local BM25

L'outil `interroger_normes` interroge un corpus de fichiers Markdown dans `Backend/knowledge_base/` contenant des extraits de DTU français, dimensions standards de mobilier et règles d'accessibilité.

BM25 a été préféré à un index vectoriel dense (FAISS, ChromaDB) pour trois raisons : **simplicité** (aucune dépendance à un modèle d’embedding, démarrage en <1 s), **prévisibilité** (correspondances par mots-clés reproductibles sur le vocabulaire technique français) et **efficacité** sur un corpus court (quelques centaines de paragraphes, où la latence d’un appel d’embedding serait supérieure au gain en pertinence).

3.7 Persistance MongoDB

MongoDB est lancé via [docker-compose](#) et expose **huit collections** dans la base `house_design`.

Collection	Contenu
<code>houses</code>	Maisons enregistrées (id, position, type, métadonnées).
<code>rooms</code>	Pièces dessinées par l’IA (murs, sols).
<code>placements</code>	Objets posés — statut <code>pending/delivered/deleted</code> .
<code>teleports</code>	Historique des téléportations.
<code>scene_objects</code>	Indice consolidé pour <code>obtenir_etat_scene</code> .
<code>events</code>	Journal d’événements pour analytics.
<code>sessions</code>	Métadonnées de session (durée, comptes).
<code>ui_commands</code>	File des commandes UI (open/close dashboard).

Table 3.4. Les huit collections MongoDB

3.8 Protocole Unity Backend

Le protocole repose sur du **polling HTTP** à intervalle court (1,5–2 s) plutôt que sur WebSocket. Ce choix est justifié par la résilience : le Wi-Fi du Quest est instable, particulièrement en mode Air Link. Une connexion WebSocket exigerait une logique de reconnexion avec risque de perte de messages. Le polling est nativement résilient : si une requête échoue, la suivante la remplace, sans état à synchroniser.

Le coût réseau est négligeable : 3 requêtes GET toutes les 2 s, ≈ 200 octets chacune, soit ≈ 300 octets/s comprimés en GZip — plusieurs ordres de grandeur en-dessous du débit d’un flux vidéo Air Link.

Le pattern **ACK explicite** (publish $\rightarrow$ poll $\rightarrow$ instantiate $\rightarrow$ confirm) garantit l'idempotence : si une instantiation Unity échoue silencieusement, le document reste **pending** et sera retenté à la prochaine itération.

4. Vitrine fonctionnelle

4.1 Contrôle vocal en français

Le contrôle vocal est la modalité principale d'interaction. L'utilisateur maintient la touche **V** (clavier) ou le bouton **B** (Quest) enfoncée pour parler. L'approche **push-to-talk** a été préférée à la VAD pour deux raisons : absence de faux-positifs sur bruits ambiants, et contrôle explicite du timing.

[Capture du joueur en train de parler (vue PC ou caméra externe Quest)]

[VoiceRecorder.cs](#) gère toute la chaîne côté Unity : capture micro (16kHz mono, `Microphone.Start`), conversion WAV, envoi multipart, attente de réponse, décodage MP3 base64, lecture via [AudioSource](#), émission des événements de pipeline.

Support de l'interruption

Si l'utilisateur appuie à nouveau sur **V** pendant que l'avatar est en Réflexion ou en Parole, [VoiceRecorder](#) coupe la requête HTTP en cours via un [CancellationToken](#), stoppe l'AudioSource TTS et émet [OnRecordingStart](#) — l'avatar repasse en Attente et enregistre la nouvelle phrase.

4.2 Avatar IA animé

L'avatar est un humanoïde **Mixamo Ch41** — mesh FBX rigué squelettiquement, hiérarchie standard Hips → Spine → Chest → Neck → Head.

[Avatar dans la scène MaMaison, état Idle, vu de face]

4.2.1 Machine d'états d'animation

L'[Animator](#) (`AvatarController.controller`) gère **cinq états** enchaînés par des Triggers Unity :

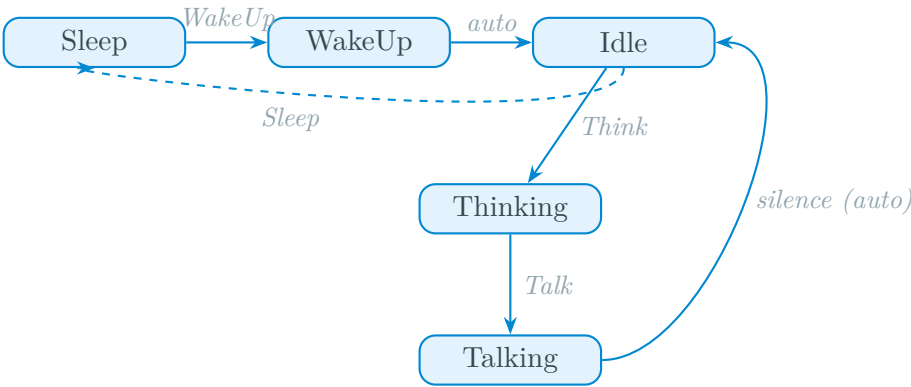


Figure 4.1. Machine d'états d'animation de l'avatar

Hook	Moment exact	État résultant
OnRecordingStart	Touche V enfoncée	WakeUp → Idle
OnSendStart	Audio envoyé, attente	Thinking
OnTtsStart(audio clip)	MP3 reçu et décodé	Talking
OnTtsEnd	Clip audio terminé	Idle

Table 4.1. Hooks de synchronisation avatar / pipeline vocal

4.3 Mains animées dual-mode VR / PC

Les mains visibles proviennent du package **Animated Hands** : modèles bleus stylisés, seize os par main, blend-tree à trois poses (default, fist, pinch).

[Mains VR — état grip relâché et grip serré côte à côte]

- **Mode VR — XR Input** : position, rotation et valeurs grip/trigger lues via `InputDevices` à chaque frame. Saisie native par `XRDirectInteractor`.
- **Mode PC** : mains attachées en local à la caméra (offset calibré sur 60° FOV). Saisie par `OverlapSphere` + touche J/H.

4.4 Téléportation vocale

La téléportation vocale permet de couvrir cinquante mètres en une seconde sans cinétose.

1. L'utilisateur prononce une instruction de téléportation.
2. Le LLM appelle `teleporter_joueur(destination="maison 3")`.
3. `tools.py` résout la destination en coordonnées (houses / rooms / placements).

4. Un document `{status: "pending"}` est inséré dans `teleports`.
5. `TeleportReceiver.cs` poll et détecte le document.
6. Fondu noir ($0 \rightarrow 1$ sur 0,5s) $\rightarrow$ déplacement $\rightarrow$ fondu retour ($1 \rightarrow 0$ sur 0,5s).
7. ACK via POST `/api/teleport/{id}/confirm`.

Résultat

Durée totale : $\approx 1,2$ seconde, transition douce, aucune cinétose.

4.5 Placement intelligent d'objets

L'instruction « mets une chaise » déclenche `placer_objet(object_name="chaise")`, qui insère dans `placements` :

```

1 {
2   "_id": "obj_chaise_20260530143012",
3   "house_id": "maison_001",
4   "prefab": "Furniture/Chaise/Chaise_1",
5   "world_x": 13.7,    // BUILD_AHEAD (3m devant le joueur)
6   "world_y": 0.0,    // remis a zero par le raycast Unity
7   "world_z": -1.0,
8   "rotation_y": 326.0, // dos tourne au joueur (angle + 180)
9   "status": "pending"
10 }
```

Listing 4.1. Document de placement inséré dans MongoDB

`PrefabPlacer.cs` effectue cinq étapes : résolution du prefab, raycast au sol, instanciation, ajout des composants d'interaction VR, ACK serveur.

[Placement d'une chaise devant le joueur (avant/après)]

4.6 Tableau de bord analytique

Le tableau de bord affiche sans quitter la VR : commandes traitées, placements, téléportations, latence moyenne, durée de session.

[Dashboard ouvert dans la vue VR — panneau WorldSpace en haut à droite]

L'intention d'ouverture est détectée **avant le LLM** via une regex (latence : ≈ 450 ms contre 3–9s pour un appel complet à l'agent). Le canvas se ferme automatiquement dès que l'utilisateur appuie sur V pour reparer.

4.7 Génération de maison guidée

Au-delà du placement d'objets, l'agent peut construire des **maisons entières** à partir de descriptions en langage naturel. « Construis-moi une maison à trois chambres » dé-

clenche une séquence : `initialiser_maison` puis `ajouter_piece_unity` pour chaque pièce. Le rendu final repose sur `ModularHousePack1` — l’agent sélectionne le modèle le plus adapté et le place en une commande.

[Vue d’ensemble de la scène avec une maison générée par l’IA]

5. Stack technique

Couche	Technologie	Version	Justification
Client VR	Unity LTS	2022.3	Compatibilité Quest 2/3, XR Toolkit mature.
XR	XR Interaction Toolkit + Oculus XR Plugin	3.5	Saisie native, support Quest standalone.
Backend	FastAPI + Uvicorn	0.110/0.27	Async natif, Pydantic, OpenAPI auto.
Agent LLM	agno	latest	Tool-calling déclaratif, retry intégré.
LLM principal	Groq Llama 3.3 70B	—	Quota gratuit le plus généreux, latence <1 s.
LLM secondaire	Groq Llama 3.1 8B	—	Quota TPM séparé, latence très faible.
LLM tertiaire	Gemini 2.5 Flash-Lite	—	Repli indépendant de Groq.
STT principal	Deepgram Nova-3	—	200 h gratuites/mois, ≈300 ms en fr.
STT secondaire	Groq Whisper Large v3	—	18 000 s-audio/h gratuites.
TTS	Edge-TTS (Azure)	latest	Gratuit, sans clé, voix DeniseNeural.
RAG	rank_bm25	latest	Local, démarrage instantané, pas d'embedding.
Base de données	MongoDB 7	7	Documents semi-structurés, Docker one-shot.
Avatar	Mixamo Ch41	—	Rig Humanoid, animations Asset Store.
Mains VR	Animated Hands	Asset Store	Stylisés Oculus, blend-tree Grip/Trigger.

(suite)

Conteneurisation	Docker Compose v2	v2	MongoDB en un fichier YAML.
-------------------------	-------------------	----	-----------------------------

Table 5.1. Stack technique complète

6. Conclusion

6.1 Synthèse des accomplissements

Archi-Agent VR démontre qu'il est possible de bâtir, sur du matériel grand public et avec des APIs gratuites, une expérience de conception architecturale en VR pilotée intégralement par la voix en français. Sept fonctionnalités majeures sont opérationnelles : contrôle vocal push-to-talk à 300 ms, avatar animé synchronisé, mains VR riggées, téléportation en fondu noir, placement intelligent parmi 50 prefabs, tableau de bord analytique en WorldSpace, et génération de maisons à partir de descriptions naturelles.

6.2 Limites actuelles

- **Quotas free-tier** — Une utilisation intensive peut épuiser les quotas TPM de Groq en une dizaine de minutes ; la cascade tertiaire Gemini absorbe le pic mais avec une latence supérieure.
- **Latence cumulée** — Lorsque le niveau 1 d'un service doit expirer avant que le niveau 2 prenne le relais, la latence perçue peut atteindre 15 s — au-dessus du seuil d'inconfort en VR.
- **Dépendance Wi-Fi** — L'usage en standalone Quest nécessite un Wi-Fi domestique stable.
- **Catalogue fini** — Seules les 50 entrées de `spawnable_objects.json` sont reconnues.

6.3 Perspectives

- **LLM auto-hébergé** — Un modèle 7–13B quantisé (Llama 3 8B AWQ, Qwen 2.5 7B) via vLLM ou llama.cpp éliminerait toute dépendance externe.
- **Mixed Reality pass-through** — L'API pass-through du Quest 3 permettrait de superposer les meubles virtuels à l'environnement physique.
- **Mode multi-utilisateur** — MongoDB sur serveur partagé + backend FastAPI exposé sur Internet permettraient à plusieurs casques de collaborer dans la même maison.
- **Génération procédurale fine** — ProBuilder ou Houdini Engine permettraient à l'agent de produire des meubles uniques à la demande.
- **Évaluation utilisateur formelle** — Étude comparative entre Archi-Agent VR et un logiciel CAO traditionnel sur des tâches d'agencement simples.

A. Référence des endpoints REST

Méthode	Endpoint	Rôle
POST	/api/chat/audio	Voix → STT → LLM → TTS, retourne MP3 base64 + texte.
GET	/api/placements/pending	Placements non encore acquittés par Unity.
GET	/api/placements?house_id={id}	Tous les placements delivered (replay au démarrage).
POST	/api/placements/{id}/confirm	ACK d'un placement instancié.
GET	/api/teleport/pending	Téléportations non encore acquittées.
POST	/api/teleport/{id}/confirm	ACK d'une téléportation.
GET	/api/ui/pending	Commandes UI (open/close dashboard).
GET	/api/analytics/dashboard	Statistiques de session.
GET	/api/status	Liveness check + provider LLM courant.

Table A.1. Endpoints REST du backend

B. Variables d'environnement

```
1 LLM_API_KEY      = Cle Groq pour le LLM principal (compte 1)
2 GROQ_API_KEY     = Cle Groq pour Whisper STT (compte 2, quota
   separe)
3 GEMINI_API_KEY   = Cle Google AI Studio pour le repli Gemini
4 DEEPGRAM_API_KEY = Cle Deepgram pour STT principal
5 MONGO_URI        = URI MongoDB (default : mongodb://localhost
   :27017/)
```

Listing B.1. Fichier .env (depuis .env.example)

C. Structure du dépôt

```
1  Unity_project/
2  |-- Backend/
3  |   |-- api.py                # FastAPI app, endpoints
4  |   |-- llm_agent.py          # Agents Groq + Gemini, prompt
      systeme
5  |   |-- voice.py              # Cascade STT + TTS Edge-TTS
6  |   |-- tools.py              # Les 8 outils declares a l'
      agent
7  |   |-- rag.py                # BM25 sur knowledge_base/
8  |   |-- database.py           # MongoDB helpers
9  |   |-- analytics.py          # Stats de session, events
10 |   |-- spatial.py             # Calcul de position devant le
      joueur
11 |   |-- prefab_catalog.py      # Lecture de spawnable_objects
      .json
12 |   |-- knowledge_base/        # Corpus normatif (.md)
13 |   |-- spawnable_objects.json # Catalogue prefabs
14 |   |-- docker-compose.yml     # MongoDB
15 |   |-- requirements.txt
16 |   +-- .env.example
17 +-- Unity/
18     +-- Assets/
19         |-- Scenes/MaMaison.unity
20         |-- Scripts/
21             |-- VoiceRecorder.cs
22             |-- HandPresence.cs
23             |-- PrefabPlacer.cs
24             |-- TeleportReceiver.cs
25             |-- DashboardController.cs
26             |-- UICommandPoller.cs
27             |-- AvatarEventBridge.cs
28             +-- LayoutReceiver.cs
29         |-- Avatar/              # FBX Mixamo Ch41 + Animator
30         |-- Animated Hands/      # Prefabs mains, controllers
31     +-- Resources/
32         |-- Furniture/           # Prefabs mobilier
33         +-- Houses/              # ModularHousePack1
```

Listing C.1. Arborescence du projet